

СОДЕРЖАНИЕ

Введение	2
1 Исследование предметной области.....	4
1.1 Актуальность проблемы	4
1.2 Обзор существующих систем мониторинга Greenplum.....	5
1.3 Особенности мониторинга Greenplum	8
1.4 Формирование требований к разрабатываемой системе	12
1.5 Выводы по главе	13
2 Выбор инструментов и технологий для реализации системы мониторинга	14
2.1 Система сбора и хранения метрик	14
2.2 Архитектура разрабатываемой системы мониторинга	16
2.3 Организация сбора метрик	18
2.4 Визуализация метрик в Grafana.....	19
2.5 Организация системы оповещений	25
2.6 Практическое использование разработанной системы	27
2.7 Выводы по главе	29
3 Тестирование и апробация системы	29
3.1 Описание тестовой среды	30
3.2 Нагрузочное тестирование системы.....	30
3.3 Проверка системы оповещений.....	32
3.4 Анализ результатов тестирования.....	33
3.5 Выводы по главе	34
Заключение	35
Список использованных источников.....	36
Приложение 1 Исходный код разработанной системы	37

ВВЕДЕНИЕ

Актуальность темы исследования. В современных информационных системах ключевую и неотъемлемую роль играют базы данных, обеспечивая надежное хранение, обработку и бесперебойный доступ к данным. От стабильности и производительности системы управления базами данных (СУБД) напрямую зависит корректность выполнения операций, скорость обработки пользовательских запросов и общая доступность информационных ресурсов.

В условиях постоянного роста объема данных и стремительно возрастающей нагрузки на СУБД, особую значимость приобретает задача эффективного мониторинга состояния СУБД.

Надёжная работа СУБД напрямую зависит от своевременного выявления проблем, связанных с производительностью, доступностью компонентов и использованием вычислительных ресурсов. Возникновение подобных отклонений может привести к критическому замедлению работы системы или к полной потере доступа к данным. В этой связи системы мониторинга становятся неотъемлемой частью инфраструктуры, обеспечивая постоянный контроль состояния базы данных и быстрое реагирование на сбои в работе системы.

Таким образом, актуальность настоящего исследования обусловлена отсутствием бесплатного комплексного решения для мониторинга СУБД Greenplum, обладающего необходимым набором функций для оперативного контроля состояния кластера. Существующие инструменты реализованы в рамках коммерческих продуктов, что ограничивает их доступность, и обуславливает необходимость разработки альтернативного решения.

Объектом исследования является процесс мониторинга технического состояния, производительности и доступности распределённой СУБД Greenplum.

Предмет исследования – технологии и программные средства автоматизированного мониторинга СУБД Greenplum, обеспечивающие контроль за ключевыми метриками производительности и формирование системы оповещений о критических инцидентах.

Цель данной работы – разработка комплексной системы мониторинга и оповещений для СУБД Greenplum.

Для достижения поставленной цели были поставлены следующие **задачи работы**:

- A. Исследовать предметную область, изучить существующие решения и определить их достоинства и недостатки;
- B. Выбрать необходимые инструменты и технологии для реализации системы мониторинга;
- C. Спроектировать и реализовать систему мониторинга;
- D. Провести тестирование и апробацию разработанного решения.

1 ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

Данная глава посвящена исследованию предметной области.

В параграфе 1.1 рассматривается актуальность применения СУБД Greenplum в современных аналитических системах, а также обосновывается необходимость разработки системы мониторинга. Параграф 1.2 посвящён анализу существующих решений для мониторинга Greenplum, в частности коммерческой системы Greenplum Command Center, и выявлению её основных функциональных ограничений. В параграфе 1.3 рассматриваются теоретические основы мониторинга СУБД, а также особенности мониторинга Greenplum, включая рекомендации разработчиков по проведению основных мониторинговых процедур. Параграф 1.4 содержит формирование требований к разрабатываемой системе мониторинга на основе проведённого анализа предметной области и существующих решений.

1.1 Актуальность проблемы

В настоящее время СУБД Greenplum получила достаточно широкое распространение в качестве основного хранилища данных и аналитических систем. Это обусловлено несколькими факторами.

Greenplum построен на архитектуре массово-параллельной обработки (MPP), изображенной на рис.1.1. Это означает, что выполнение запросов распределяется между несколькими вычислительными узлами, что позволяет значительно увеличить скорость обработки данных и обеспечить горизонтальную масштабируемость.

MPP Architecture

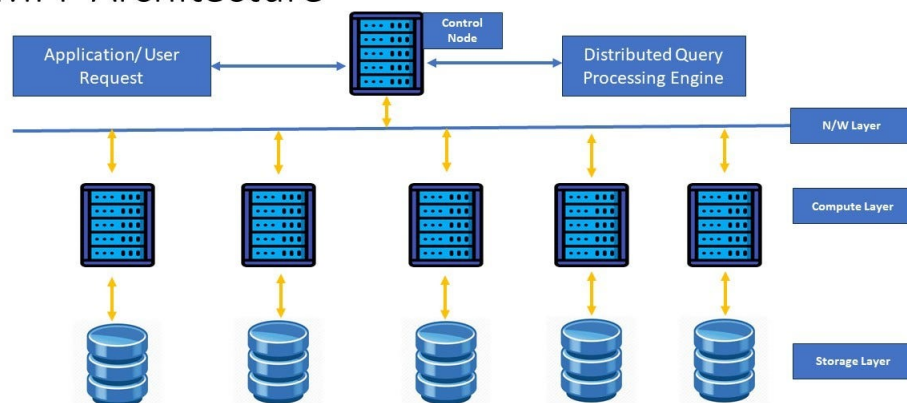


Рис.1.1. MPP Архитектура

За счет распределения данных по сегментам и параллельного выполнения запросов на сегментах Greenplum показывает лучшую производительность на больших объемах данных по сравнению с традиционными реляционными СУБД, такими как PostgreSQL.

Использование зеркальных (mirror) сегментов обеспечивает повышение отказоустойчивости системы, поскольку при выходе из строя основного сегмента выполнение операций автоматически переключается на соответствующий резервный сегмент без прерывания работы всего кластера.

Несмотря на высокую производительность, масштабируемость и встроенные механизмы обеспечения отказоустойчивости, эксплуатация Greenplum требует постоянного контроля состояния системы. В условиях промышленной эксплуатации даже кратковременные отклонения в работе СУБД могут привести к снижению общей производительности кластера или возникновению критических ошибок. По этой причине задача мониторинга состояния СУБД является неотъемлемой частью обязанностей администратора базы данных.

На данный момент представлено только одно специализированное решение для мониторинга СУБД Greenplum. Это коммерческое решение – Greenplum Command Center. Данный продукт является компонентом коммерческого дистрибутива Tanzu VMware Greenplum. Конкретная стоимость дистрибутива в открытом доступе не сообщается. Подробное рассмотрение данной системы будет приведено в следующем параграфе 1.2.

Доступных на территории Российской Федерации бесплатных комплексных решений, обеспечивающих мониторинг Greenplum в настоящее время не представлено.

Приведенные факторы подтверждают актуальность и практическую значимость данной работы.

1.2 Обзор существующих систем мониторинга Greenplum

Greenplum Command Center является инструментом мониторинга и управления кластером СУБД Greenplum, входящим в состав коммерческого дистрибутива Greenplum. Командный центр позволяет отслеживать показатели производительности системы, анализировать работоспособность кластера, а так же позволяет выполнять задачи администрирования кластера.

Система предоставляет веб-интерфейс, позволяющий в режиме реального времени отслеживать ключевые системные показатели работы кластера, включая загрузку вычислительных узлов, использование оперативной памяти, нагрузку на процессор, а также активность выполнения SQL-запросов.

В открытых источниках есть примеры интерфейса системы, который содержит несколько дашбордов. Дашборд изображенный на рис.1.2, отображает основные показатели производительности кластера. На отдельном дашборде, изображенном на рис.1.3 доступен мониторинг активности запросов к базе данных.

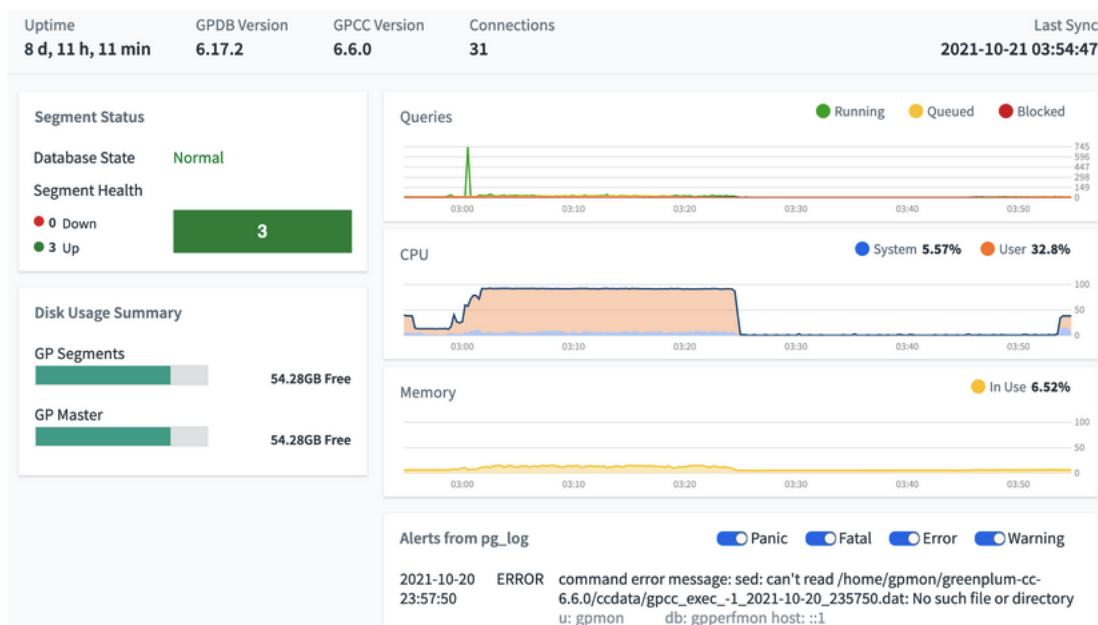


Рис.1.2. Дашборд системных метрик в Greenplum Command Center

Dashboard

Query Monitor

Host Metrics

Cluster Metrics

History

Table Browser

Recommendations

Workload

System

Admin

Query Monitor

Current queries by all users

2021-05-24 03:10:23

Help

Queries

Sessions

2 Running

0 Queued

1 Blocked

Advanced Search (0)

Pause Query Monitor

CANCEL QUERY

EXPORT

☐	Query ID	Status	User	Database	Workload	Submitted	Queued Time	Run Time	Spill Files	CPU Time
☐	1621409718-26753-3	Blocked	gpmon	gpmon	admin_group	20:28:37	3d 6h 41m 53s	--	--	--
☐	1621409718-37684-2	Running	gpadmin	gpadmin	default_group	08:16:50	1d 23h 59m 13s	18h 54m 27s	--	2s
☐	1621409718-37687-2	Running	gpadmin	gpadmin	default_group	08:16:55	1d 23h 59m 15s	18h 54m 20s	--	2s

Рис.1.3. Дашборд активности запросов в Greenplum Command Center

В системе настроен определенный список событий, при возникновении которых пользователю отправляется уведомление, посмотреть которое можно в специальной вкладке «Уведомления», пример которой можно увидеть на рис.1.4. Получение уведомлений на электронную почту требует отдельной настройки со стороны администратора системы.

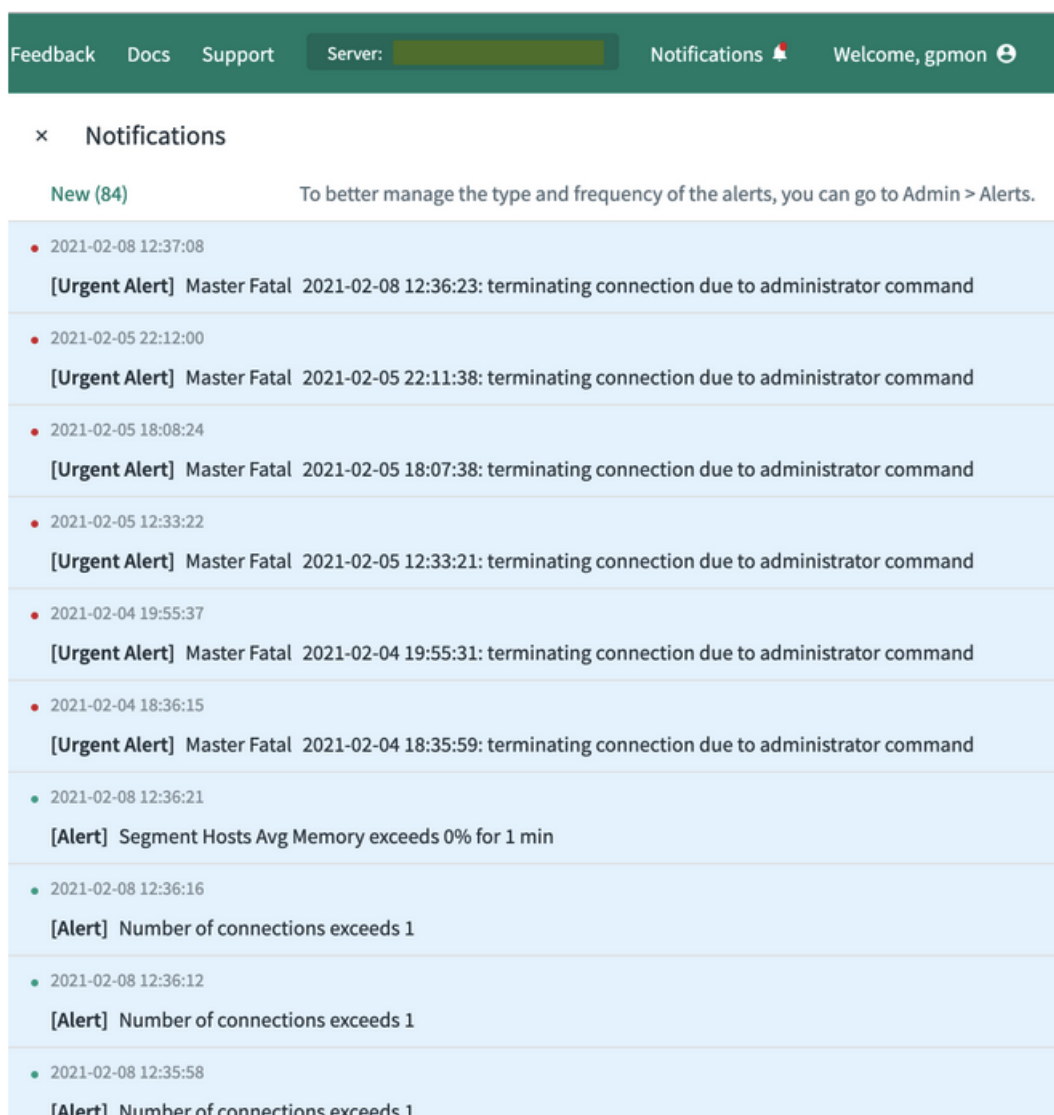


Рис.1.4. Пример уведомлений в Greenplum Command Center

Несмотря на достаточно широкий функционал, данный инструмент имеет ряд недостатков. В первую очередь он распространяется как часть коммерческого продукта и требует использования дорогостоящей лицензии. Кроме того встроенная система уведомлений без дополнительной настройки позволяет узнавать о критических инцидентах только непосредственно из веб-интерфейса командного центра, что препятствует оперативному реагированию на внештатные ситуации.

Таким образом, Greenplum Command Center представляет собой коммерческое решение для мониторинга СУБД Greenplum, недоступное широкому кругу пользователей, что подтверждает необходимость разработки альтернативных систем, ориентированных на доступность, и простоту эксплуатации.

1.3 Особенности мониторинга Greenplum

Мониторинг СУБД представляет собой процесс сбора и анализа информации о состоянии системы с целью обеспечения её стабильной и эффективной работы. Он включает в себя отслеживание различных показателей, характеризующих использование ресурсов и выполнение операций.

К основным задачам мониторинга относятся [7]:

- контроль системных показателей, таких как загрузка процессора, использование памяти и дисковых ресурсов;
- отслеживание доступности системы;
- анализ выполняемых запросов и их длительности;
- выявление узких мест в производительности;
- контроль ключевых показателей работы СУБД.

Реализация мониторинга требует опоры на техническую документацию и регламенты администрирования системы. В связи с этим теоретической основой формирования требований к системе мониторинга в рамках данной работы послужили рекомендации разработчиков Greenplum. В документации изложен перечень регламентных мониторинговых процедур, направленных на обеспечение высокой доступности и стабильной производительности кластера.

Основные рекомендованные процедуры мониторинга можно разделить на 4 основных группы:

1. Мониторинг состояния кластера Greenplum

Таблица 1.1

Рекомендованные процедуры проверки состояния кластера Greenplum

Процедура	Необходимое действие	Степень значимости	Рекомендуемая частота проверки
Проверка работоспособности и доступности мастер-сегмента	Необходимо выполнить запрос: SELECT count(*) FROM gp_segment_configuration;	Критическая	Каждые 5-10 минут
Проверка списка недоступных сегментов	Необходимо выполнить запрос: SELECT * FROM gp_segment_configuration WHERE status = 'd';	Высокая	Каждые 5-10 минут

Процедура	Необходимое действие	Степень значимости	Рекомендуемая частота проверки
Проверка активных, но не синхронизированных сегментов	Необходимо выполнить запрос: SELECT * FROM gp_segment_configuration WHERE mode = 'n' and status = 'u' and content <> -1;	Высокая	Каждые 5-10 минут
Проверка распределённого выполнения запроса	Необходимо выполнить запрос: SELECT gp_segment_id, count(*) FROM gp_dist_random('pg_class') GROUP BY 1; Для каждого сегмента должна быть возвращена 1 строка	Критическая	Каждые 5-10 минут
Проверка состояния репликации master	Необходимо psql команду: psql <dbname> -c 'SELECT pid, state FROM pg_stat_replication;' Значение state должно быть «STREAMING»	Критическая	Каждые 5-10 минут

2. Мониторинг аппаратного обеспечения и операционной системы

Таблица 1.2

Рекомендованные процедуры проверки аппаратного обеспечения и операционной системы

Процедура	Необходимое действие	Степень значимости	Рекомендуемая частота проверки
Проверка использования дискового пространства на томах, используемых для хранения данных базы данных Greenplum	Необходимо проверить наличие свободного дискового пространства. Рекомендуется предпринять меры при достижении 75% занятого объема диска	Критическая	Каждые 10-30 минут
Проверка состояния дисков RAID массива	Совместно с администраторами сервера необходимо проверить состояние всех дисков RAID массива. При наличии ошибок или выходе из строя каких-либо аппаратных компонент незамедлительно заменить их	Критическая	При создании кластера, или при наличии признаков неисправности

Процедура	Необходимое действие	Степень значимости	Рекомендуемая частота проверки
Проверка состояния сетевых интерфейсов	Необходимо проверить сетевые интерфейсы на наличие ошибок и потерянных пакетов передачи данных	Высокая	Каждый час

3. Мониторинг состояния каталогов

Таблица 1.3

Рекомендованные процедуры проверки состояния каталогов

Процедура	Необходимое действие	Степень значимости	Рекомендуемая частота проверки
Проверка согласованности каталогов каждой базы данных	Необходимо убедиться, что каталог на каждом узле кластера согласован и находится в исправном состоянии. Для этого рекомендуется использовать утилиту gpcheckcat: gpcheckcat -O	Высокая	Каждую неделю
Проверка целостности системного каталога	Рекомендуется использовать утилиту gpcheckcat: gpcheckcat -R pgclass для выявления несогласованных записей в системных таблицах pg_class и pg_attribute	Критическая	Каждый месяц
Проверка корректности схем и временных пространств имен	Необходимо убедиться в отсутствии некорректных временных схем и отсутствующих определений пространств имен в системном каталоге. Для этого рекомендуется использовать утилиту gpcheckcat: gpcheckcat -R namespace	Высокая	Каждый месяц
Проверка зависимостей системного каталога	Необходимо убедиться в отсутствии зависимостей от несуществующих объектов в системном каталоге базы данных. Для этого рекомендуется использовать утилиту gpcheckcat: gpcheckcat -R dependency	Высокая	Каждый месяц

4. Мониторинг состояния базы данных

Таблица 1.4

Рекомендованные процедуры проверки состояния базы данных

Процедура	Необходимое действие	Степень значимости	Рекомендуемая частота проверки
Проверка таблиц с отсутствующей статистикой	Необходимо проверять таблицы, для которых отсутствует актуальная статистика. Для этого рекомендуется выполнить запрос к представлению <code>gp_toolkit.gp_stats_missing</code> : <code>SELECT * FROM gp_toolkit.gp_stats_missing;</code>	Средняя	После загрузки большого объема данных
Проверка таблиц с избыточным неиспользуемым пространством	Необходимо выявлять таблицы, содержащие значительный объем неиспользуемого пространства, не освобождаемого обычной очисткой. Для этого рекомендуется выполнить запрос к представлению <code>gp_toolkit.gp_bloat_diag</code> : <code>SELECT * FROM gp_toolkit.gp_bloat_diag;</code>	Средняя	Каждую неделю или каждый месяц
Очистка таблиц от удаленных строк	Необходимо освобождать пространство, занимаемое удаленными строками, для его повторного использования. Для этого рекомендуется выполнять команду <code>VACUUM <table></code> .	Высокая	Каждый день
Обновление статистики таблиц	Необходимо поддерживать статистику таблиц в актуальном состоянии для корректной работы оптимизатора запросов. Для этого рекомендуется использовать утилиту <code>analyze</code> : <code>analyze <database> -a.</code>	Высокая	После загрузки большого объема данных
Резервное копирование данных базы данных	Необходимо создавать актуальные резервные копии мастер-узла и сегментов кластера для возможности восстановления базы данных в случае отказа. Для этого рекомендуется использовать утилиту <code>gpbackup</code> .	Критическая	Каждый день

Процедура	Необходимое действие	Степень значимости	Рекомендуемая частота проверки
Обслуживание системных каталогов	Необходимо выполнять очистку, переиндексацию и обновление статистики системных таблиц для поддержания эффективной работы системного каталога. Для этого рекомендуется использовать команды VACUUM, REINDEX SYSTEM и утилиту analyzedb -s pg_catalog.	Высокая	Каждую неделю

Представленные рекомендации демонстрируют, что обеспечение стабильной работы Greenplum требует контроля большого числа параметров. Однако значительная часть перечисленных процедур относится к периодическому регламентному обслуживанию, предполагает участие администратора и, соответственно, выходит за рамки разрабатываемой системы мониторинга.

Для разработки автоматизированной системы мониторинга наибольший интерес представляют процедуры непрерывного оперативного контроля, позволяющие своевременно фиксировать отклонения в работе СУБД.

В рамках данной работы был выбран подход, основанный на анализе показателей, доступных на мастер-узле кластера. Такой подход позволяет получить обобщенное представление о текущем состоянии системы, включая сведения о нагрузке, доступности сегментов и возникновении критических ошибок, а также сокращает сложность архитектуры разрабатываемого решения, поскольку отпадает необходимость установки дополнительных компонентов на каждый вычислительный сегмент. При этом доступных на управляющем узле данных достаточно для выявления большинства критических ситуаций, связанных с производительностью, активностью и доступностью СУБД. Таким образом, выбранный подход обеспечивает баланс между полнотой собираемой диагностической информации и простотой практической реализации системы мониторинга.

1.4 Формирование требований к разрабатываемой системе

На основании проведённого анализа существующего решения Greenplum Command Center, а также рассмотренных особенностей мониторинга СУБД

Greenplum, можно сформулировать основные требования к разрабатываемой системе мониторинга:

- обеспечение централизованного сбора и хранения диагностической информации о состоянии кластера. К числу контролируемых параметров должны относиться системные метрики вычислительного узла, показатели доступности сегментов, сведения о нагрузке на базу данных, а также информация о возникновении критических ошибок в работе СУБД;
- наличие информативного дашборда, отображающего основные показатели СУБД;
- наличие механизма автоматического оповещения посредством электронной почты о критических инцидентах в работе СУБД;
- обеспечение доступности и простоты внедрения разрабатываемого решения. Система не должна требовать использования коммерческих лицензий, установки большого числа вспомогательных компонентов или сложной процедуры интеграции в существующую инфраструктуру.

1.5 Выводы по главе

В рамках данной главы была рассмотрена предметная область мониторинга СУБД Greenplum. Проведен анализ существующего решения Greenplum Command Center, рассмотрены его основные функциональные возможности и выявлены ключевые недостатки.

Также были рассмотрены теоретические аспекты мониторинга СУБД. На основе которых сформулированы требования к разрабатываемой системе мониторинга, определяющие её функциональные возможности.

Полученные результаты составляют теоретическую и практическую основу для дальнейшей разработки архитектуры и реализации системы мониторинга Greenplum.

2 ВЫБОР ИНСТРУМЕНТОВ И ТЕХНОЛОГИЙ ДЛЯ РЕАЛИЗАЦИИ СИСТЕМЫ МОНИТОРИНГА

2.1 Система сбора и хранения метрик

Как показано в предыдущей главе, система сбора и хранения метрик является основной составляющей разрабатываемого решения мониторинга.

На основании требований, сформулированных в предыдущей главе, определим основные критерии, предъявляемые к инструменту автоматизированного сбора и хранения метрик:

- возможность регулярного автоматизированного сбора метрик из различных источников, включая системные представления Greenplum и средства мониторинга операционной системы;
- поддержка хранения временных рядов с сохранением истории изменений показателей;
- наличие встроенных механизмов агрегации, фильтрации и обработки поступающих метрик;
- совместимость с внешними системами визуализации и оповещения;
- простота настройки и администрирования;
- отсутствие необходимости использования коммерческой лицензии.

Среди наиболее распространенных свободно распространяемых инструментов, удовлетворяющих указанным требованиям, можно выделить Zabbix и Prometheus. Оба решения широко применяются для построения систем мониторинга и позволяют реализовать сбор, хранение и анализ метрик, а также интегрируются со средствами визуализации и оповещения.

Рассмотрим основные компоненты архитектуры Zabbix.

Zabbix Server – центральный элемент системы, который выполняет прием данных от подключенных узлов, их обработку, сохранение в базе данных. Кроме того, сервер отвечает за формирование уведомлений и выполнение настроенных сценариев оповещения.

Zabbix Agent обеспечивает сбор информации о состоянии отслеживаемых хостов и передачу собранных значений на сервер мониторинга.

Zabbix Frontend предоставляет веб-интерфейс управления мониторингом. Через него выполняется настройка, и конфигурирование системы, а также построение дашбордов.

Для хранения накопленных метрик в Zabbix используется внешняя реляционная база данных. В качестве системы хранения могут использоваться PostgreSQL, MySQL, Oracle и другие поддерживаемые СУБД.

Также в системе реализован встроенный механизм оповещений, поддерживающий отправку уведомлений по различным каналам связи.

Таким образом, Zabbix представляет собой универсальную многофункциональную платформу инфраструктурного мониторинга.

Несмотря на широкие функциональные возможности Zabbix, его использование в рамках разрабатываемой системы мониторинга СУБД Greenplum имеет ряд существенных ограничений.

Основным недостатком является отсутствие готовых специализированных плагинов для работы с СУБД Greenplum. Для получения диагностических показателей в Zabbix потребовалась бы разработка алгоритма сбора метрик, специфических для Greenplum. При расширении списка собираемых метрик это приводит к существенному увеличению трудоёмкости сопровождения и усложняет администрирование системы.

Еще одной особенностью инструмента является способ хранения данных. В Zabbix – используется внешняя реляционная база данных, что ограничивает возможности оперативной аналитической обработки временных рядов. В частности, в Zabbix отсутствует специализированный язык запросов, аналогичный PromQL, предназначенный для гибкой агрегации, фильтрации и вычисления производных метрик.

Перейдем к рассмотрению Prometheus. Prometheus представляет собой комплексное решение для сбора, хранения и анализа метрик, разработанное специально для задач инфраструктурного и сервисного мониторинга.

Архитектурно Prometheus включает несколько основных компонентов.

Центральным элементом является Prometheus Server, который выполняет периодический опрос источников метрик по протоколу HTTP, сохраняет полученные значения во встроенное хранилище временных рядов TSDB (Time Series Database), а также обеспечивает выполнение запросов к накопленным данным.

Источниками метрик в архитектуре Prometheus выступают экспортеры – вспомогательные программные модули, публикующие показатели работы наблюдаемой системы в формате, понятном Prometheus. Сервер мониторинга регулярно обращается к экспортерам по pull-модели, считывая актуальные значения показа-

телей. Такой подход упрощает управление процессом сбора данных и позволяет легко масштабировать систему при добавлении новых контролируемых узлов.

Основой модели данных в Prometheus является метрика, представляющая собой временной ряд. Каждая метрика определяется как набор значений, изменяющихся во времени, и хранится в виде последовательности пар «временная метка – значение».

Особенностью модели данных Prometheus является использование меток (labels), представляющих собой набор пар «ключ – значение», ассоциированных с каждой метрикой. Метки позволяют однозначно идентифицировать временной ряд и задавать дополнительные измерения, такие как имя хоста, имя базы данных, идентификатор сегмента или другие характеристики контролируемого объекта.

Таким образом, каждая метрика в Prometheus определяется уникальной комбинацией имени метрики и набора меток. Такой подход обеспечивает высокую гибкость при организации мониторинга, позволяя агрегировать, фильтровать и анализировать данные по различным срезам.

Нотация метрики в Prometheus имеет следующий вид:

```
<metric name>{<label name>=«label value» ...}
```

Пример метрики pg_locks_count:

```
pg_locks_count{datname="postgres mode="accesssharelock"}
```

Для обработки накопленных временных рядов в Prometheus используется специализированный язык запросов PromQL (Prometheus Query Language). Он предоставляет возможности фильтрации, агрегации, вычисления скоростей прироста и других производных расчетов. Это особенно важно при анализе метрик, когда требуется не только фиксировать абсолютные значения, но и выявлять динамику изменения показателей.

2.2 Архитектура разрабатываемой системы мониторинга

В рамках данной работы была разработана система мониторинга производительности СУБД Greenplum, реализованная с использованием контейнеризации на основе Docker. Такой подход позволяет обеспечить изоляцию компонентов, упростить развертывание системы и повысить её воспроизводимость.

Развёртывание системы осуществляется с использованием инструмента Docker Compose, который позволяет описать конфигурацию всех компонентов в одном файле и запускать их в рамках единой инфраструктуры.

В состав разработанной системы входят следующие компоненты:

- СУБД Greenplum, используемая в качестве источника данных для мониторинга. В тестовой конфигурации развернут кластер, включающий управляющий узел и четыре сегмента без репликации;
- Prometheus — система сбора и хранения метрик, осуществляющая периодический опрос источников данных;
- Grafana — инструмент визуализации, используемый для построения дашбордов и анализа метрик;
- Alertmanager — компонент, отвечающий за обработку и отправку уведомлений о критических событиях [2];
- Node Exporter — сервис для сбора системных метрик (нагрузка на процессор, использование памяти, дисковая активность) [5];
- Postgres Exporter — инструмент для получения метрик, связанных с работой СУБД PostgreSQL и Greenplum на уровне базы данных [1];
- Greenplum Exporter — специализированный компонент, обеспечивающий сбор метрик, специфичных для СУБД Greenplum.

Все перечисленные компоненты развёрнуты в отдельных контейнерах и взаимодействуют между собой через единую Docker-сеть. Такой подход обеспечивает модульность системы и упрощает её масштабирование и сопровождение.

Процесс развертывания системы автоматизирован с использованием конфигурационного файла `docker-compose.yml` и файла переменных окружения `.env`. Файл `.env` содержит параметры конфигурации, такие как учетные данные, адреса сервисов и другие настройки, необходимые для корректной работы системы. Перед запуском системы пользователь должен заполнить данный файл, после чего выполнить команду запуска Docker Compose. В результате все компоненты автоматически инициализируются и начинают взаимодействие.

Сбор метрик осуществляется Prometheus, который опрашивает экспортеры по заданным HTTP-эндпоинтам. Полученные данные сохраняются в виде временных рядов и используются для последующего анализа и визуализации.

Визуализация метрик реализована в Grafana. В рамках системы настроено подключение к Prometheus в качестве основного источника данных. Кроме того, реализовано дополнительное прямое подключение Grafana к базе данных Greenplum. Это позволяет выполнять SQL-запросы непосредственно из интерфейса Grafana и отображать результаты в виде табличных виджетов. Такой подход расширяет

возможности анализа и позволяет получать более детализированную информацию о состоянии системы.

Alertmanager используется для обработки событий, связанных с превышением пороговых значений метрик. Он получает сигналы от Prometheus и выполняет отправку уведомлений в соответствии с заданными правилами.

Разработанная архитектура обеспечивает централизованный сбор, хранение и визуализацию метрик, а также реализацию механизма оповещения. Использование контейнеризации и автоматизированного развертывания делает систему удобной в использовании и адаптируемой к различным условиям эксплуатации.

2.3 Организация сбора метрик

В разработанной системе мониторинга ключевую роль в процессе сбора и хранения метрик выполняет система Prometheus. Она представляет собой специализированное решение для работы с временными рядами, предназначенное для получения, хранения и обработки метрик в распределённых системах.

Архитектура Prometheus построена по принципу автономности и включает в себя все необходимые компоненты: механизм сбора данных, встроенное хранилище и язык запросов для анализа. Это позволяет использовать систему без привлечения внешних средств хранения и упрощает её развертывание и сопровождение.

Сбор метрик в Prometheus реализован по pull-модели, при которой система самостоятельно инициирует опрос источников данных через заданные интервалы времени. В качестве источников метрик в данной работе используются специализированные сервисы — экспортеры. Они выполняют преобразование внутренних показателей системы в формат, поддерживаемый Prometheus, и предоставляют их через HTTP-интерфейс.

В разработанной системе используются следующие экспортеры:

- Node Exporter — обеспечивает сбор системных метрик, включая загрузку процессора, использование оперативной памяти, состояние дисковой подсистемы и сетевую активность;
- Postgres Exporter — предоставляет метрики, характеризующие работу базы данных, такие как количество активных подключений, статистика выполнения запросов и общее состояние СУБД;
- Greenplum Exporter — дополняет набор метрик за счёт показателей, специфичных для СУБД Greenplum.

Каждый экспортер функционирует как отдельный сервис и предоставляет метрики по HTTP-эндпоинту. Prometheus обращается к этим эндпоинтам, извлекает данные и сохраняет их во внутреннем хранилище. Взаимодействие между компонентами осуществляется в рамках одной Docker-сети, что позволяет использовать внутренние имена контейнеров при настройке подключения.

Конфигурация сбора метрик задаётся в файле настроек Prometheus, где определяется список целей (targets), соответствующих адресам экспортеров, а также параметры опроса, включая интервал сбора данных. Это обеспечивает централизованное управление процессом мониторинга и позволяет гибко изменять состав источников метрик.

Полученные данные сохраняются в виде временных рядов, где каждая метрика представлена последовательностью значений, привязанных ко времени. Дополнительно используются метки (labels), позволяющие описывать характеристики метрик и выполнять их агрегацию по различным признакам. Такой подход обеспечивает гибкость при анализе данных и позволяет формировать сложные запросы.

Сбор метрик в рамках данной работы осуществляется с управляющего узла Greenplum, что позволяет получать агрегированную информацию о состоянии системы, активности подключений и использовании ресурсов. Несмотря на ограниченность источника данных, такой подход обеспечивает достаточную информативность для выявления большинства критических ситуаций.

Таким образом, использование Prometheus в сочетании с набором экспортеров позволяет реализовать централизованную систему сбора метрик, обеспечивающую регулярное получение данных, их хранение в виде временных рядов и подготовку к последующему анализу и визуализации.

2.4 Визуализация метрик в Grafana

После организации сбора и хранения метрик следующим этапом является их визуализация, позволяющая анализировать состояние системы в наглядной форме. В разработанной системе для этих целей используется Grafana — современный инструмент визуализации данных, предназначенный для работы с временными рядами.

Grafana предоставляет возможность подключения к различным источникам данных и построения интерактивных дашбордов. В рамках данной работы в

качестве основного источника данных используется Prometheus, из которого извлекаются собранные метрики. Подключение осуществляется через встроенный механизм Data Source, позволяющий выполнять запросы к системе мониторинга с использованием языка PromQL.

Одним из ключевых преимуществ Grafana является гибкость при построении дашбордов. Пользователь может создавать панели различных типов, включая графики, диаграммы, индикаторы и таблицы. Это позволяет адаптировать отображение данных под конкретные задачи мониторинга и упростить восприятие информации.

В разработанной системе был создан пользовательский дашборд, предназначенный для отображения ключевых показателей состояния СУБД и используемых ресурсов. Дашборд объединяет данные из различных источников и обеспечивает централизованное представление информации о работе системы. При его проектировании основное внимание уделялось наглядности, информативности и удобству восприятия данных.

Структурно дашборд разделён на четыре логических блока, каждый из которых отвечает за отдельный аспект мониторинга системы. Такое разделение позволяет упростить анализ и быстрее ориентироваться в показателях.

На дашборде представлены панели, отражающие текущее состояние системы. В частности, реализованы индикаторы, позволяющие оценить доступность системы и её основных компонентов. Такие элементы обеспечивают быстрое понимание текущего состояния СУБД без необходимости детального анализа графиков.

Отдельный блок дашборда посвящён мониторингу нагрузки на систему. В нём отображаются показатели использования процессора и оперативной памяти, полученные с помощью Node Exporter. Эти метрики позволяют оценить уровень загрузки системы и выявить возможные проблемы, связанные с нехваткой ресурсов.

Также на дашборде представлены метрики, характеризующие работу базы данных. К ним относятся показатели количества активных подключений, активности запросов и общего состояния СУБД. Данные метрики позволяют анализировать поведение пользователей и выявлять аномалии в работе системы.

Дополнительно используются показатели, специфичные для Greenplum, включая информацию о состоянии сегментов и их активности. Это позволяет оценивать общее состояние распределённой системы и контролировать её функционирование.

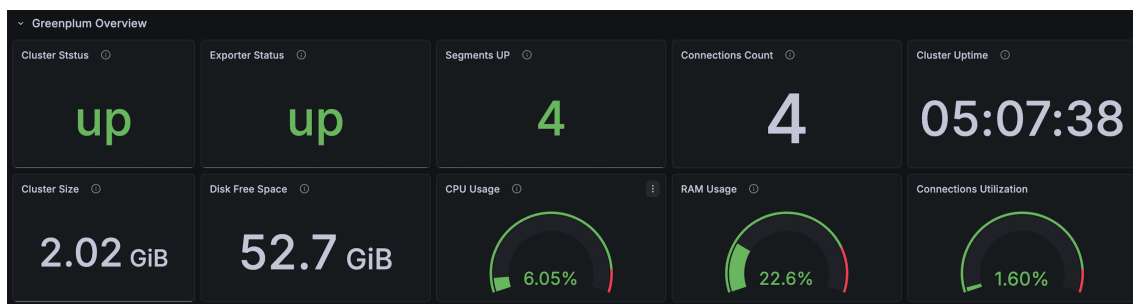


Рис.2.1. Блок "Greenplum Overview"

Первый блок "Greenplum Overview" изображенный на рис.2.1 содержит метрики, отражающие общее состояние системы и её ключевых компонентов:

- Cluster status — $\min(\text{pg_up}\{\})$. Показывает общее состояние кластера. Значение определяется по наименее доступному компоненту.
- Exporter status — $\min(\text{up}\{\})$. Отражает доступность компонентов системы мониторинга.
- Segments up — $\sum(\text{greenplum_node_segment_status}\{\text{content} \neq "-1"\})$. Показывает количество активных сегментов кластера.
- Connections count — $\sum(\text{pg_stat_database_numbackends}\{\text{job}="greenplum"\})$. Характеризует текущее количество активных подключений к базе данных.
- Cluster uptime — $\text{greenplum_cluster_uptime}$. Отражает время непрерывной работы системы с момента последнего запуска.
- Cluster size — $\sum \text{by}(\text{job})(\text{pg_database_size_bytes})$. Показывает суммарный объём данных, хранящихся в базе данных.
- Disk free space — $\text{node_filesystem_avail_bytes}$. Характеризует объём доступного дискового пространства.
- CPU usage — $(1 - \text{avg by}(\text{instance}, \text{job})(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}="idle"\}[5\text{m}]))) * 100$. Отражает уровень загрузки процессора.
- RAM usage — $(1 - (\text{node_memory_MemAvailable_bytes} / \text{node_memory_MemTotal_bytes})) * 100$. Показывает степень использования оперативной памяти.
- Connections utilization — $\sum(\text{pg_stat_database_numbackends}) / \sum(\text{pg_settings_max_connections}) * 100$. Характеризует уровень использования доступных подключений.

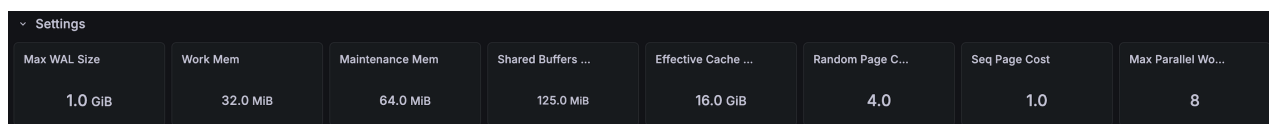


Рис.2.2. Блок "Settings"

Второй блок "Settings" изображенный на рис.2.2 содержит параметры конфигурации СУБД Greenplum:

- Max WAL size — `pg_settings_max_wal_size_bytes{}`. Определяет максимальный размер журнала предзаписи (WAL), при достижении которого инициируется контрольная точка.
- Work mem — `pg_settings_work_mem_bytes{}`. Задаёт объём памяти, выделяемой для выполнения операций сортировки и хеширования в рамках одного запроса.
- Maintenance mem — `pg_settings_maintenance_work_mem_bytes{}`. Определяет объём памяти, используемый для служебных операций, таких как вакуумизация и создание индексов.
- Shared buffers size — `pg_settings_shared_buffers_bytes{}`. Показывает объём общей буферной памяти, используемой СУБД для кэширования данных.
- Effective cache size — `pg_settings_effective_cache_size_bytes{}`. Оценивает объём доступного кэша (включая системный), используемый планировщиком запросов при выборе стратегии выполнения.
- Random page cost — `pg_settings_random_page_cost{}`. Определяет относительную стоимость случайного доступа к страницам данных.
- Seq page cost — `pg_settings_seq_page_cost{}`. Задаёт стоимость последовательного чтения страниц данных.
- Max parallel workers — `pg_settings_max_parallel_workers{}`. Определяет максимальное количество параллельных рабочих процессов, доступных для выполнения запросов.

Третий блок на рис.2.3 включает метрики, характеризующие активность базы данных и эффективность выполнения операций:

- TPS (commits) — `rate(pg_stat_database_xact_commit{datname=~"$Database"}[$__rate_interval])`. Показывает количество успешно зафиксированных транзакций в секунду.
- TPS (rollbacks) — `rate(pg_stat_database_xact_rollback{datname=~"$Database"}[$__rate_interval])`. Показывает количество откатанных транзакций в секунду.
- Fetched data — `rate(pg_stat_database_tup_fetched{datname=~"$Database"}[$__rate_interval])`. Показывает количество строк, извлечённых из базы данных.

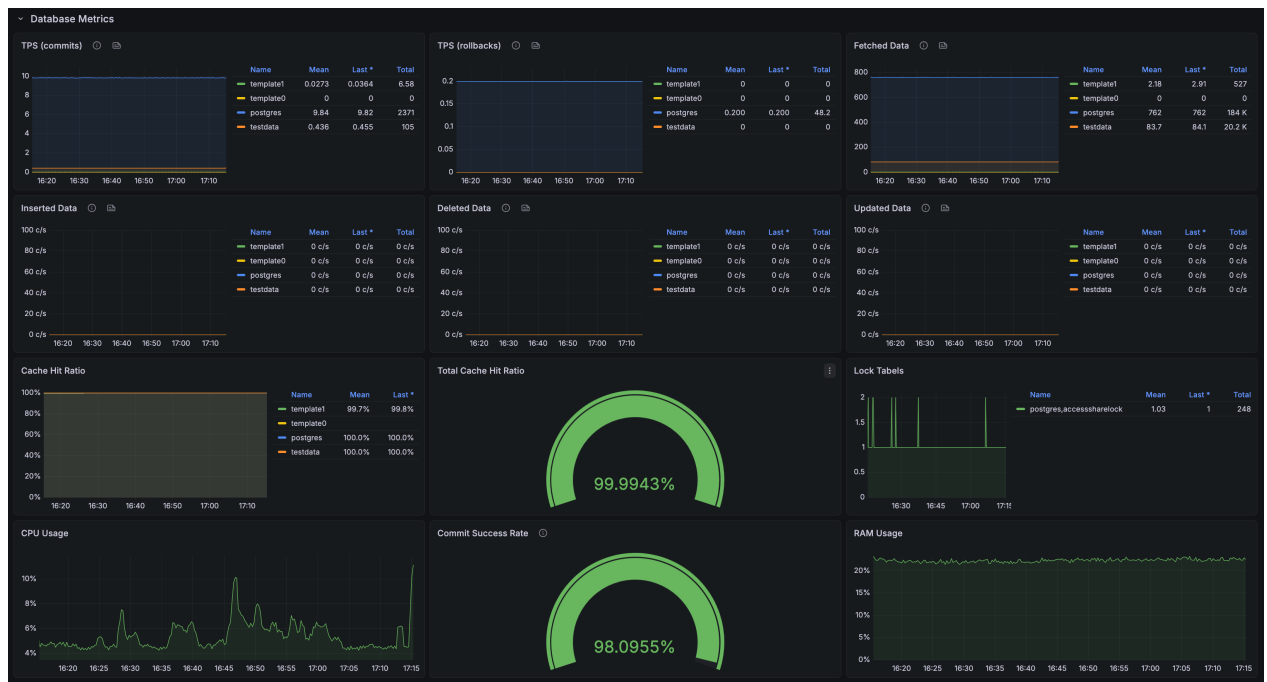


Рис.2.3. Блок "Database Metrics"

- Inserted data — `rate(pg_stat_database_tup_inserted{datname=~"$Database"}[$__rate_interval])`. Показывает интенсивность операций вставки данных.
- Deleted data — `rate(pg_stat_database_tup_deleted{datname=~"$Database"}[$__rate_interval])`. Показывает количество удалённых строк.
- Updated data — `rate(pg_stat_database_tup_updated{datname=~"$Database"}[$__rate_interval])`. Показывает количество обновлённых строк.
- Cache hit ratio — $\text{pg_stat_database_blks_hit}\{\text{datname}=\sim\text{"$Database"}\} / (\text{pg_stat_database_blks_read}\{\text{datname}=\sim\text{"$Database"}\} + \text{pg_stat_database_blks_hit}\{\text{datname}=\sim\text{"$Database"}\}) * 100$. Характеризует долю обращений к данным, обслуженных из кэша.
- Total cache hit ratio — `greenplum_server_database_hit_cache_percent_rate`. Показывает агрегированный показатель эффективности использования кэша.
- Lock tables — `pg_locks_count{datname=~"$Database"} != 0`. Отражает наличие блокировок в базе данных.
- CPU usage — $(1 - \text{avg by(instance, job)(rate(node_cpu_seconds_total\{mode="idle"\}[$__rate_interval]))}) * 100$. Показывает уровень загрузки процессора.

- Commit success rate — `greenplum_server_database_transition_commit_percent_rate`. Показывает долю успешно завершённых транзакций.
- RAM usage — $(1 - (\text{node_memory_MemAvailable_bytes} / \text{node_memory_MemTotal_bytes})) * 100$. Показывает уровень использования оперативной памяти.

schema	table_name	relkind	table_size	seq_scans	idx_scans	inserts	updates	deletes	last_vacuum	last_autovacuum	last_analyze	last_autoanalyze
public	lineitem	r	1143 MB	0	0	0	0	0				
public	orders	r	252 MB	0	0	0	0	0				
public	partsupp	r	170 MB	0	0	0	0	0				
public	customer	r	67 MB	0	0	0	0	0				
public	part	r	39 MB	0	0	0	0	0				
public	supplier	r	2848 KB	0	0	0	0	0				
public	nation	r	416 KB	0	0	0	0	0				
public	region	r	352 KB	0	0	0	0	0				
TOTAL				0	0	0	0	0				

Рис.2.4. Блок "Tables Information"

Четвёртый блок представляет собой табличное представление данных (рис.2.4), сформированное на основе SQL-запроса к системным каталогам базы данных, как показано в листинге П1.1. Данный блок предназначен для детального анализа состояния таблиц и их активности.

Описание полей табличного представления:

- Schema — схема, к которой относится таблица.
- Table name — имя таблицы.
- Relkind — тип объекта (в данном случае отображаются только обычные таблицы).
- Table size — общий размер таблицы с учётом индексов и вспомогательных данных.
- Seq scans — количество последовательных сканирований таблицы.
- Idx scans — количество обращений к таблице с использованием индексов.
- Inserts — количество операций вставки строк.
- Updates — количество операций обновления строк.
- Deletes — количество операций удаления строк.
- Last vacuum — время последнего ручного выполнения VACUUM.
- Last autovacuum — время последнего автоматического выполнения VACUUM.
- Last analyze — время последнего ручного выполнения ANALYZE.

- Last autoanalyze — время последнего автоматического выполнения ANALYZE.

Данный блок позволяет выявлять наиболее нагруженные таблицы, анализировать характер доступа к данным (последовательное или индексное сканирование), а также контролировать актуальность статистики и выполнение операций обслуживания. Это обеспечивает более глубокий уровень мониторинга по сравнению с агрегированными метриками и позволяет принимать обоснованные решения по оптимизации структуры и производительности базы данных.

2.5 Организация системы оповещений

Важным элементом системы мониторинга является механизм оповещений, позволяющий своевременно реагировать на отклонения в работе СУБД. В разработанной системе данная функциональность реализована с использованием компонента Alertmanager, входящего в экосистему Prometheus.

Alertmanager предназначен для обработки уведомлений, формируемых системой мониторинга на основе заданных правил. Он обеспечивает агрегацию, фильтрацию и маршрутизацию оповещений, а также предотвращает дублирование сообщений при множественных срабатываниях одного и того же инцидента. Дополнительно реализуется механизм группировки событий, позволяющий объединять связанные алерты в единое уведомление и тем самым снижать информационную нагрузку на пользователя.

Формирование оповещений осуществляется на стороне Prometheus с использованием правил (alerting rules), которые задаются в конфигурационном файле. Каждое правило описывает условие срабатывания на основе выражений языка PromQL. При выполнении заданного условия формируется событие, которое передаётся в Alertmanager для дальнейшей обработки.

В рамках разработанной системы все правила оповещений объединены в единый набор *greenplum-alerts* изображенный на рис.2.5, включающий 11 правил, которые охватывают ключевые аспекты работы системы: доступность компонентов, использование вычислительных ресурсов, состояние базы данных и эффективность выполнения запросов.

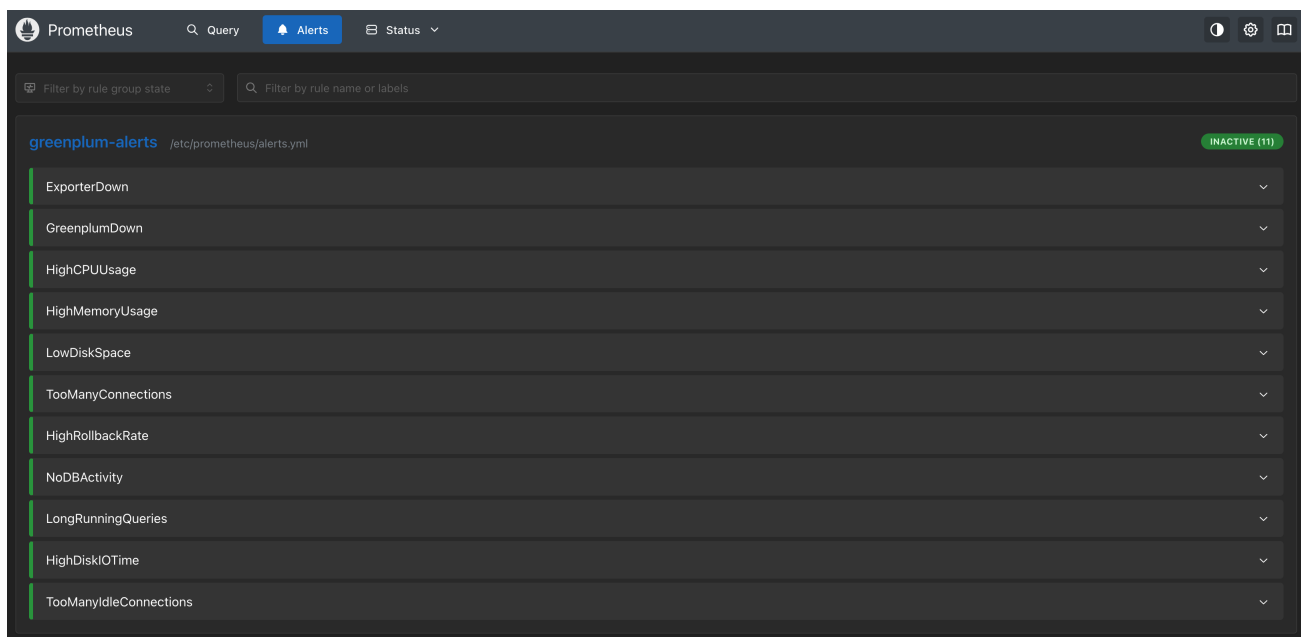


Рис.2.5. Список правил формирования оповещений

С точки зрения логической структуры, все оповещения можно разделить на несколько групп:

- Оповещения доступности компонентов — контролируют состояние системы мониторинга и СУБД. К ним относятся *ExporterDown* и *GreenplumDown*, которые фиксируют недоступность соответствующих сервисов.
- Оповещения использования системных ресурсов — отслеживают нагрузку на инфраструктуру, включая процессор, оперативную память и дисковое пространство. В данную группу входят *HighCPUUsage*, *HighMemoryUsage*, *LowDiskSpace* и *HighDiskIOTime*.
- Оповещения состояния базы данных — характеризуют активность и поведение СУБД, включая количество подключений и транзакционную активность. К ним относятся *TooManyConnections*, *TooManyIdleConnections* и *NoDBActivity*.
- Оповещения производительности и качества выполнения запросов — позволяют выявлять потенциальные проблемы в работе запросов и транзакций. В данную группу входят *HighRollbackRate* и *LongRunningQueries*.

Каждое правило оповещения содержит три основных параметра: условие срабатывания (*expr*), время устойчивого нарушения условия (*for*) и уровень критичности (*labels.severity*). Использование параметра *for* позволяет исключить кратковременные всплески метрик и формировать уведомления только при устойчивом отклонении значений.

Примером может служить правило контроля загрузки процессора (рис.2.6), при котором формируется предупреждение, если использование CPU превышает 85% в течение более чем 2 минут. Аналогично, при превышении порога использования оперативной памяти выше 90% формируется критическое уведомление.

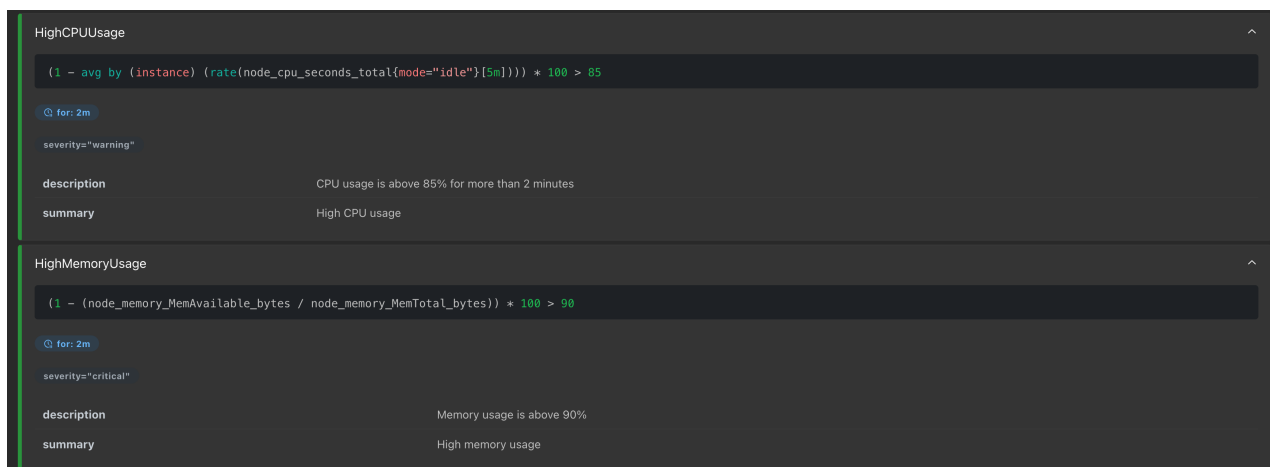


Рис.2.6. Примеры правил для оповещений *HighCPUUsage*, *HighMemoryUsage*

Alertmanager выполняет маршрутизацию уведомлений на основе заданных правил. В данной работе настроена отправка сообщений на электронную почту, что обеспечивает централизованный контроль состояния системы. Также используется механизм группировки оповещений, который объединяет связанные события и снижает количество отдельных уведомлений.

Таким образом, разработанная система оповещений обеспечивает автоматизированный контроль состояния СУБД Greenplum, позволяя своевременно выявлять критические и потенциально опасные ситуации, связанные с доступностью компонентов, нагрузкой на систему и эффективностью выполнения запросов. Это повышает общую надёжность и устойчивость функционирования системы мониторинга.

2.6 Практическое использование разработанной системы

Разработанная система мониторинга ориентирована на практическое применение и предполагает минимальное участие пользователя в процессе настройки и развертывания. При проектировании особое внимание уделялось автоматизации конфигурации и упрощению запуска системы.

Процесс использования системы включает несколько последовательных этапов. На первом этапе пользователю необходимо заполнить файл переменных

окружения `.env`, в котором задаются основные параметры конфигурации. К ним относятся данные для подключения к СУБД Greenplum (адрес, порт, учетные данные), параметры сетевых портов сервисов, настройки доступа к Grafana, а также параметры почтового сервера для отправки уведомлений.

Пример структуры файла `.env` приведён в приложении П1.3:

После заполнения файла переменных окружения выполняется запуск скрипта генерации конфигурационных файлов (П1.4):

```
./scripts/generate-configs.sh
```

Данный скрипт автоматически формирует необходимые конфигурационные файлы для всех компонентов системы, включая Prometheus, Alertmanager и экспортеры, на основе заданных параметров. Это позволяет исключить ручное редактирование конфигураций и снизить вероятность ошибок при настройке.

На завершающем этапе осуществляется запуск системы с использованием Docker Compose:

```
docker compose up -d
```

Содержание файла можно посмотреть в приложении П1.2.

В результате выполнения данной команды происходит автоматическое развертывание всех компонентов системы и их инициализация. После запуска система становится полностью готовой к работе без необходимости дополнительной настройки.

Важной особенностью разработанного решения является высокий уровень автоматизации: пользователю достаточно задать исходные параметры и выполнить две команды, после чего он получает функционирующую систему мониторинга. Такой подход существенно упрощает процесс внедрения и делает систему доступной для использования без глубоких знаний в области настройки отдельных компонентов.

Практическая ценность разработанной системы заключается в том, что она представляет собой не набор разрозненных инструментов, а целостное интегрированное решение, обеспечивающее полный цикл мониторинга — от сбора метрик до генерации и доставки оповещений. Это позволяет сократить время развертывания, снизить сложность эксплуатации и повысить надёжность мониторинга СУБД Greenplum.

Таким образом, предложенный подход обеспечивает удобство использования, воспроизводимость развертывания и готовность системы к применению в реальных условиях.

2.7 Выводы по главе

В рамках данной главы была рассмотрена реализация системы мониторинга СУБД Greenplum, включающей сбор, хранение, визуализацию метрик и механизм оповещений.

Была разработана архитектура системы, основанная на использовании контейнеризации и инструментов Docker Compose, что позволило обеспечить изоляцию компонентов, упростить развертывание и повысить воспроизводимость решения. Все элементы системы интегрированы в единую инфраструктуру и взаимодействуют между собой автоматически.

Реализован механизм централизованного сбора метрик с использованием Prometheus и набора специализированных экспортеров, обеспечивающих получение как системных, так и прикладных показателей работы СУБД. Для анализа состояния системы разработан пользовательский дашборд в Grafana, включающий несколько логических блоков, отражающих различные аспекты функционирования базы данных и инфраструктуры.

Дополнительно реализована система оповещений на основе Alertmanager, включающая набор правил, позволяющих выявлять критические и потенциально опасные ситуации, связанные с доступностью компонентов, нагрузкой на систему и поведением базы данных.

Таким образом, в результате выполнения работы была создана не только архитектура системы мониторинга, но и её полноценная реализация в виде готового программного решения, обеспечивающего комплексный мониторинг СУБД Greenplum и пригодного для практического использования.

3 ТЕСТИРОВАНИЕ И АПРОБАЦИЯ СИСТЕМЫ

В рамках данной главы проводится проверка работоспособности разработанной системы мониторинга, а также её апробация в условиях, приближенных к реальной эксплуатации. Основной целью тестирования является оценка корректно-

сти сбора и визуализации метрик, а также проверка функционирования механизма оповещений при возникновении различных типов нагрузок и нестандартных ситуаций.

3.1 Описание тестовой среды

В качестве тестовой базы данных была использована синтетическая база данных стандарта TPC-H [8], предназначенная для моделирования аналитической нагрузки. Данный набор данных широко применяется для оценки производительности систем управления базами данных и позволяет воспроизводить сложные сценарии обработки запросов.

На основе TPC-H была развернута база данных в СУБД Greenplum, содержащая набор таблиц, отражающих бизнес-процессы (заказы, поставщики, клиенты, регионы и др.). Для проведения тестирования использовался стандартный набор из 22 аналитических SQL-запросов (Q1–Q22), каждый из которых характеризуется значительным объёмом вычислений, множественными соединениями таблиц и операциями агрегации.

3.2 Нагрузочное тестирование системы

Нагрузочное тестирование проводилось путём последовательного выполнения набора TPC-H запросов. Для этого был сформирован единый SQL-скрипт, содержащий все 22 запроса, выполнение которого осуществлялось с использованием утилиты `psql`. При запуске скрипта дополнительно фиксировалось время выполнения каждого запроса, что позволило оценить их ресурсоёмкость.

В ходе тестирования рассматривался сценарий последовательного выполнения запросов, позволяющий оценить базовую производительность системы и характер нагрузки, создаваемой отдельными аналитическими операциями. Такой подход обеспечивает контролируемые условия эксперимента и позволяет проследить влияние каждого запроса на состояние системы мониторинга и значения метрик.

В процессе выполнения запросов осуществлялось наблюдение за показателями, отображаемыми на дашборде Grafana. На рис.3.1, рис.3.2 и рис.3.3 было зафиксировано, что при увеличении интенсивности нагрузки наблюдается:

- значительный рост загрузки процессора (на рис.3.3);
- увеличение потребления оперативной памяти;

- повышение интенсивности операций ввода-вывода;
- рост показателей транзакционной активности (TPS) (на рис.3.2).

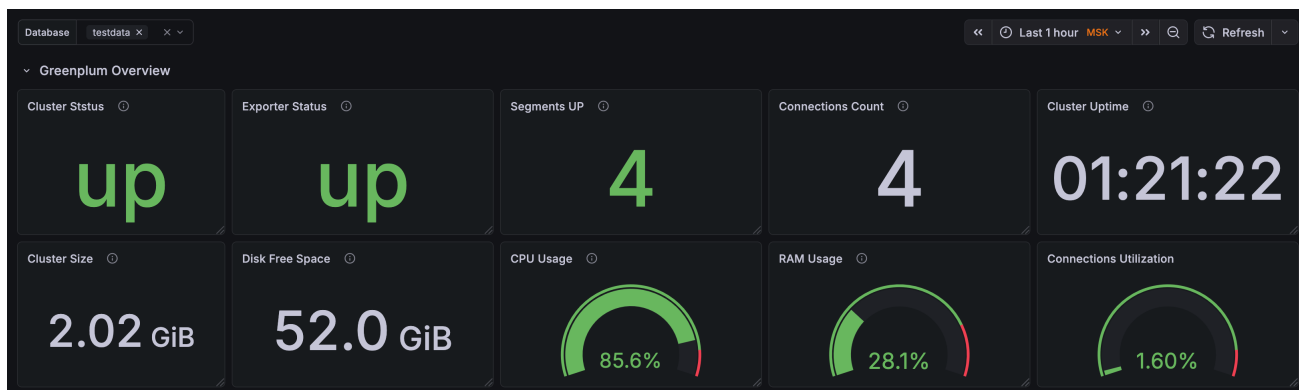


Рис.3.1. Показатели блока "Greenplum Overview" при выполнении тестовой нагрузки

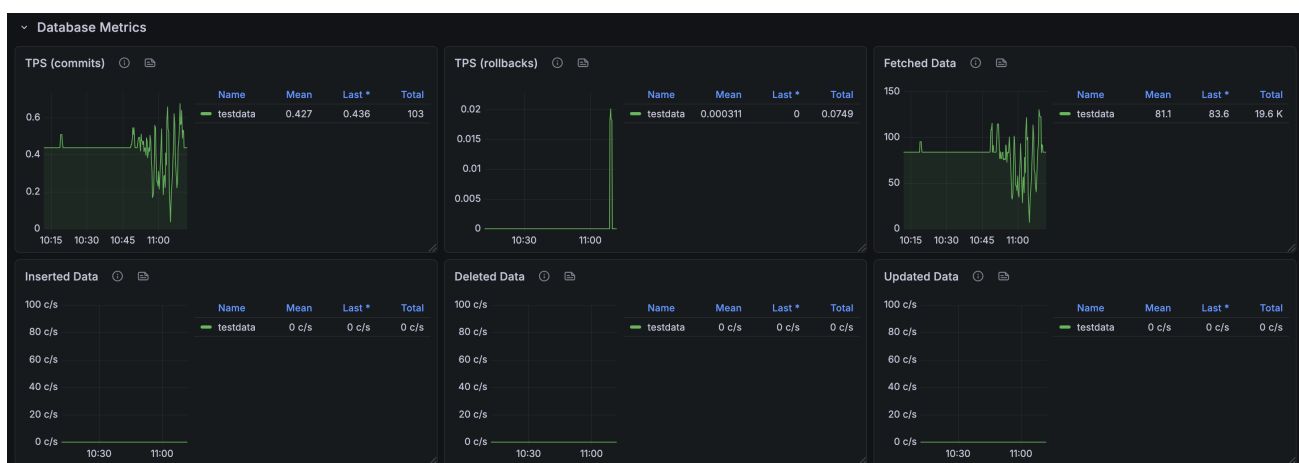


Рис.3.2. Показатели блока "Database Metrics" при выполнении тестовой нагрузки

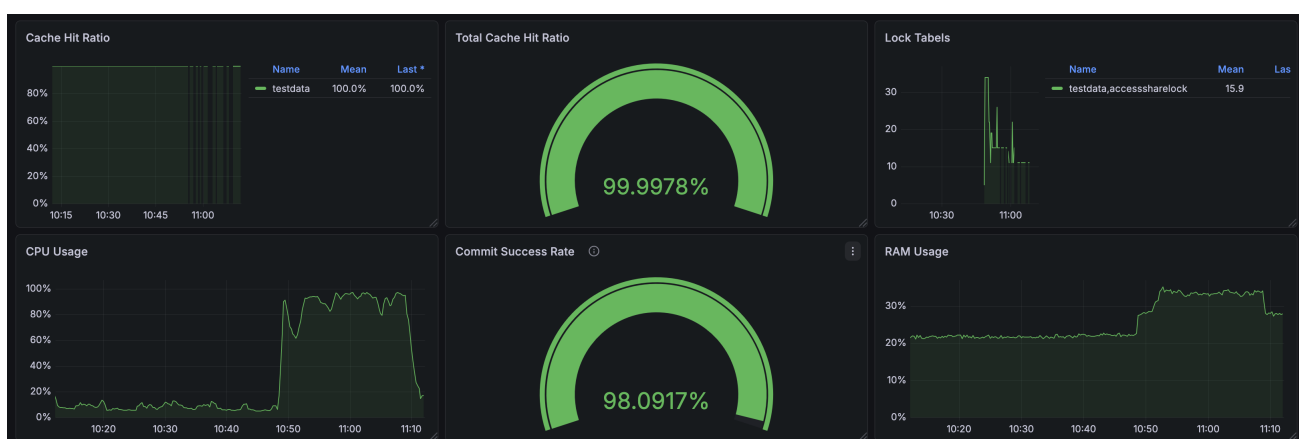


Рис.3.3. Показатели блока "Database Metrics" при выполнении тестовой нагрузки

Наиболее ресурсоёмкие запросы (например, содержащие множественные соединения и агрегации) приводили к заметным пиковым значениям загрузки CPU и дисковой подсистемы. Это подтверждает корректность работы системы мони-

торинга, так как изменения метрик адекватно отражают характер выполняемой нагрузки.

3.3 Проверка системы оповещений

Для апробации механизма оповещений были проведены тесты, направленные на искусственное создание ситуаций, при которых должны срабатывать заданные правила.

В ходе тестирования были рассмотрены следующие сценарии:

- Высокая загрузка процессора — создавалась за счёт выполнения ресурсоёмких запросов. При превышении порогового значения загрузки CPU срабатывало оповещение *HighCPUUsage*;
- Длительно выполняющиеся запросы — достигались за счёт выполнения сложных аналитических запросов. Это приводило к срабатыванию оповещения *LongRunningQueries*;
- Недоступность компонентов — моделировалась остановкой одного из контейнеров (например, экспортера), что приводило к срабатыванию оповещения *ExporterDown*.

При наступлении указанных условий система Prometheus формировала события оповещений, которые передавались в Alertmanager. Далее происходила их обработка, группировка и отправка уведомлений на заданный адрес электронной почты.

В ходе тестирования на рис.3.4, рис.3.5 и рис.3.6 было подтверждено, что уведомления формируются корректно, содержат необходимую информацию о типе события, уровне критичности и описании проблемы, а также доставляются пользователю с минимальной задержкой.

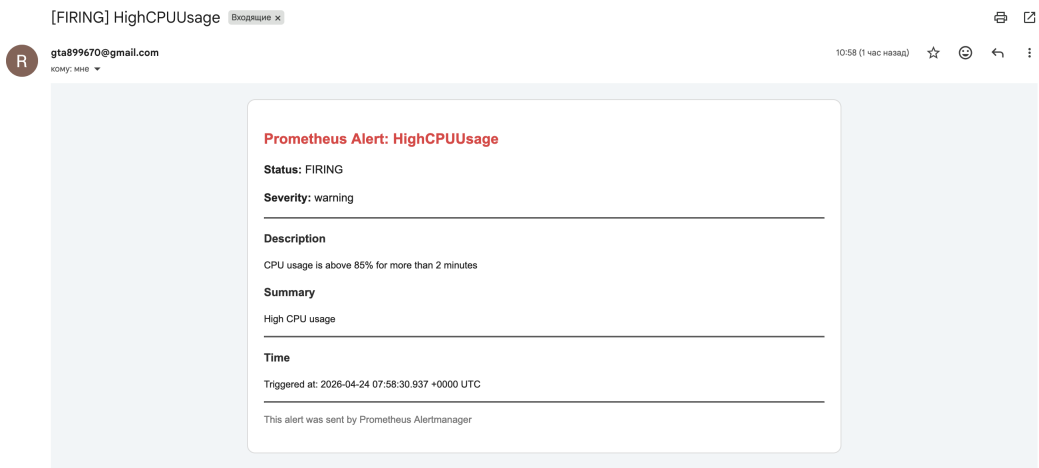


Рис.3.4. Пример оповещения *HighCPUUsage* полученный на электронную почту

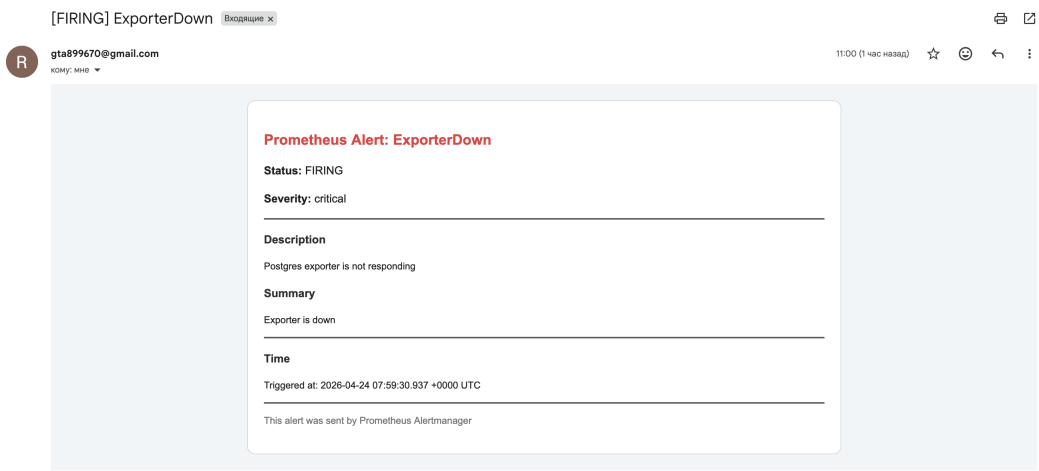


Рис.3.5. Пример оповещения *ExporterDown* полученный на электронную почту

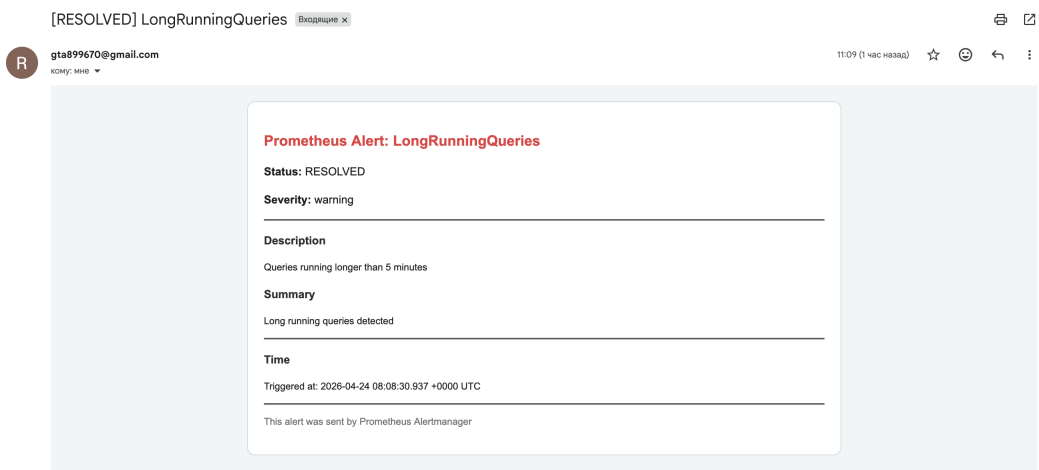


Рис.3.6. Пример оповещения *LongRunningQueries* полученный на электронную почту

3.4 Анализ результатов тестирования

Проведённые испытания показали, что разработанная система мониторинга корректно выполняет все основные функции:

- обеспечивает стабильный сбор метрик в условиях различной нагрузки;
- корректно отражает изменения состояния системы на дашборде;
- позволяет выявлять узкие места и ресурсоёмкие операции;
- своевременно реагирует на критические события посредством системы оповещений.

3.5 Выводы по главе

Использование ТРС-Н запросов в качестве нагрузки позволило продемонстрировать поведение системы в условиях, приближенных к реальным аналитическим сценариям. Это подтверждает применимость разработанного решения для мониторинга производительности СУБД Greenplum в практических задачах.

Таким образом, результаты тестирования свидетельствуют о работоспособности, устойчивости и практической значимости разработанной системы мониторинга.

ЗАКЛЮЧЕНИЕ

В рамках данной работы была решена задача разработки системы мониторинга производительности СУБД Greenplum. В результате проведённого исследования были проанализированы существующие подходы к мониторингу баз данных и выбраны современные инструменты, обеспечивающие сбор, хранение, визуализацию метрик и организацию системы оповещений.

Разработанная система реализована на основе экосистемы Prometheus и включает в себя полный набор компонентов, необходимых для комплексного мониторинга: сбор метрик с использованием экспортеров, их хранение и обработку, визуализацию в Grafana, а также механизм оповещений на базе Alertmanager. Все компоненты интегрированы в единую инфраструктуру с использованием контейнеризации Docker, что обеспечивает удобство развертывания и воспроизводимость системы.

В рамках практической части была проведена апробация разработанного решения с использованием тестовой базы данных ТРС-Н. В ходе тестирования выполнялись аналитические запросы, позволяющие создать нагрузку на систему и оценить её поведение. Результаты наблюдений показали, что разработанный дашборд в Grafana обеспечивает наглядное отображение изменений ключевых метрик, включая загрузку процессора, использование памяти и активность базы данных.

Дополнительно была проведена проверка системы оповещений. В результате искусственного создания критических ситуаций было подтверждено корректное срабатывание правил оповещений и доставка уведомлений пользователю. Это демонстрирует работоспособность и практическую применимость реализованного механизма контроля состояния системы.

Таким образом, разработанная система представляет собой не набор отдельных инструментов, а готовое интегрированное решение, обеспечивающее полный цикл мониторинга СУБД — от сбора и анализа метрик до оперативного оповещения о возникающих проблемах. Это подтверждает её практическую ценность и возможность использования в реальных условиях эксплуатации.

В качестве направлений дальнейшего развития можно выделить расширение набора метрик и правил оповещений, добавление новых каналов уведомлений, а также адаптацию системы для мониторинга других типов баз данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Postgres Exporter. — URL: https://github.com/prometheus-community/postgres_exporter (дата обращения: 06.03.2026).
2. Документация Alertmanager. — URL: <https://prometheus.io/docs/alerting/latest/alertmanager/> (дата обращения: 06.03.2026).
3. Документация Grafana. — URL: <https://grafana.com/docs/> (дата обращения: 06.03.2026).
4. Документация Greenplum. — URL: <https://techdocs.broadcom.com/us/en/vmware-tanzu/data-solutions/tanzu-greenplum/7/greenplum-database/landing-index.html> (дата обращения: 06.03.2026).
5. Документация Node Exporter. — URL: https://github.com/prometheus/node_exporter (дата обращения: 06.03.2026).
6. Документация Prometheus. — URL: <https://prometheus.io/docs/introduction/overview/> (дата обращения: 06.03.2026).
7. Лесовский А.В. Мониторинг PostgreSQL. — URL: <https://edu.postgrespro.ru/monitoring.pdf> (дата обращения: 06.03.2026).
8. Тестирование TCP-H. — URL: https://docs.starrocks.io/docs/benchmarking/TPC-H_Benchmarking/ (дата обращения: 06.03.2026).

Приложение 1

Исходный код разработанной системы

Листинг П1.1

SQL-запрос для анализа таблиц

```

SELECT
n.nspname AS schema,
c.relname AS table_name,
5 c.relkind,
pg_size_pretty(pg_total_relation_size(c.oid)) AS table_size,
COALESCE(s.seq_scan, 0) AS seq_scans,
COALESCE(s.idx_scan, 0) AS idx_scans,
COALESCE(s.n_tup_ins, 0) AS inserts,
10 COALESCE(s.n_tup_upd, 0) AS updates,
COALESCE(s.n_tup_del, 0) AS deletes,
s.last_vacuum,
s.last_autovacuum,
s.last_analyze,
15 s.last_autoanalyze
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
LEFT JOIN pg_stat_user_tables s ON s.relid = c.oid
WHERE c.relkind = 'r'
20 AND n.nspname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(c.oid) DESC;

```

Листинг П1.2

Файл docker-compose.yml

```

version: '3.9'

services:
5 prometheus:
  image: prom/prometheus:latest
  container_name: monitoring-prometheus
  volumes:
10   - ./prometheus/prometheus.yml:/etc/prometheus/prometheus
      .yml
   - ./prometheus/rules:/etc/prometheus/rules
  command:
    - --config.file=/etc/prometheus/prometheus.yml
  ports:

```



```

15     - "${PROMETHEUS_PORT}:9090"
      networks:
        - monitoring-network
      restart: unless-stopped

20 alertmanager:
      image: prom/alertmanager:latest
      container_name: monitoring-alertmanager
      volumes:
        - ./alertmanager/alertmanager.yml:/etc/alertmanager/
          alertmanager.yml
25     - ./alertmanager/template.tpl:/etc/alertmanager/
          template.tpl
      command:
        - --config.file=/etc/alertmanager/alertmanager.yml
      ports:
        - "${ALERTMANAGER_PORT}:9093"
30     networks:
        - monitoring-network
      restart: unless-stopped

grafana:
35     image: grafana/grafana:latest
      container_name: monitoring-grafana
      environment:
        - GF_SECURITY_ADMIN_USER=${GRAFANA_ADMIN_USER}
        - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD}
40     volumes:
        - ./grafana/provisioning:/etc/grafana/provisioning
        - ./grafana/dashboards:/var/lib/grafana/dashboards
      ports:
        - "${GRAFANA_PORT}:3000"
45     networks:
        - monitoring-network
      depends_on:
        - prometheus
      restart: unless-stopped

50 node-exporter:
      image: prom/node-exporter:latest
      container_name: monitoring-node-exporter
      ports:
55     - "9100:9100"
      networks:
        - monitoring-network

```

```

restart: unless-stopped

60 postgres-exporter:
  image: prometheuscommunity/postgres-exporter:master
  container_name: monitoring-postgres-exporter
  environment:
    - DATA_SOURCE_NAME=postgres://${GP_USER}:${GP_PASSWORD}
      @${GP_HOST}:${GP_PORT}/${GP_DATABASE}?sslmode=disable
65 ports:
    - "9187:9187"
  networks:
    - monitoring-network
  extra_hosts:
70   - "host.docker.internal:host-gateway"
  restart: unless-stopped

greenplum-exporter:
  image: qinyugui/greenplum-exporter:1.1
75 container_name: monitoring-greenplum-exporter
  environment:
    - GPDB_DATA_SOURCE_URL=postgres://${GP_USER}:${GP_PASSWORD}@${GP_HOST}:${GP_PORT}/${GP_DATABASE}?
      sslmode=disable
  ports:
    - "9188:9297"
80 networks:
    - monitoring-network
  extra_hosts:
    - "host.docker.internal:host-gateway"
  restart: unless-stopped
85

mkdocs:
  image: polinux/mkdocs:arm64v8-1.5.2
  container_name: monitoring-mkdocs
  volumes:
90   - ./runbooks:/docs
  ports:
    - "8000:8000"
  networks:
    - monitoring-network
95 restart: unless-stopped

networks:
  monitoring-network:
    driver: bridge

```

Листинг П1.3

Единый конфигурационный файл .env.example

```

GP_HOST=
GP_PORT=
GP_USER=
5 GP_PASSWORD=
GP_DATABASE=

EXPORTER_PORT=9187
PROMETHEUS_PORT=9090
10 ALERTMANAGER_PORT=9093
GRAFANA_PORT=3000

GRAFANA_ADMIN_USER=
GRAFANA_ADMIN_PASSWORD=

15 ALERT_EMAIL_TO=
ALERT_EMAIL_FROM=

SMTP_HOST=smtp.gmail.com
20 SMTP_PORT=587
SMTP_USER=
SMTP_PASSWORD=

```

Листинг П1.4

Файл generate-configs.sh

```

#!/bin/bash

set -e

5 echo "Generating configs..."

# Prometheus
envsubst < prometheus/prometheus.yml.template > prometheus/
  prometheus.yml

10 # Alertmanager
envsubst < alertmanager/alertmanager.yml.template >
  alertmanager/alertmanager.yml

# Grafana datasource
15 envsubst < grafana/provisioning/datasources/datasource.yml.
  template > grafana/provisioning/datasources/datasource.
  yml

```

```
echo "Configs generated"
```